Stage-Aware Group Scheduling for Offline Agent Workloads

Anonymous submission

Abstract

We study admission scheduling for offline, statically orchestrated, typed agent workflows, exposing an optimization layer between agent orchestration and token-level LLM serving. When concurrent workflows expose ready stages with compatible prompt templates, an external scheduler can group these requests to improve cross-workflow batching and prefix locality, an opportunity not explicitly considered by default request-level serving policies. We formulate this as a mixed-integer program (MIP) that groups ready stages of the same type into shared serving windows under workflow precedence and memory constraints.

The key empirical finding is that homogeneous stage grouping reduces the marginal serving-time coefficient B in the affine model $T(N) = A + B \cdot N$ by 32%–54%, while the fixed coefficient A remains nearly unchanged. Prefix-cache hit rate measurements indicate that improved exact-prefix locality is an important contributor to the lower marginal serving time.

Across five models from 7 B to 72 B, including both dense and MoE structure, matched live executions on a common vLLM backend show that MIP-guided stage scheduling reduces makespan by 7.0%–9.4% relative to default vLLM continuous batching with automatic prefix caching at a representative moderate workload, corresponding to throughput increases of 7.5%–10.2%. In controlled DeepDAG stress tests, the reduction reaches 47.0% under the tested Poisson and bursty arrival patterns. Matched WorkBench executions do not show degradation in sandbox-evaluated task outcomes. These results establish cross-workflow stage scheduling as a productive new abstraction for offline agent serving.

Introduction

Modern agent applications increasingly serve many workflow executions in parallel. A user task expands into planning, retrieval, tool calls, verification, and response synthesis, where later stages depend on earlier outputs and some workflows branch before they converge. When many such workflows run concurrently, the serving system must decide not only which request to run next, but also which ready stage from which workflow should enter the next serving window, so that stages of the same computational type can be batched together for shared prefix KV cache reuse.

Existing systems cover only parts of this problem. Agent orchestration frameworks decompose work into agents and tools, while LLM serving systems optimize batching and KV cache management once requests reach the engine. Default agent orchestration and request-level serving policies do not jointly optimize workflow precedence and typed-stage compatibility across concurrent workflows. This leaves a missing layer in which stages of the same computational type become available across workflows at overlapping times, yet conventional FCFS fragments them into separate windows and continuous batching mixes heterogeneous stage types, leaving prefix reuse opportunities unclaimed. We focus on offline, statically orchestrated agent workflows. A complete batch of workflow instances is available before execution, and each workflow follows a typed orchestration graph whose stage identities, prompt templates, and precedence relations are declared by the application. The scheduler therefore does not infer an agent's future action from model outputs. It only orders and groups LLM calls associated with stages that are already specified by the workflow graph and have become ready at runtime. This setting arises when benchmark tasks are executed through predefined agent pipelines, as well as in batched rollout generation and scheduled enterprise workflows whose orchestration graphs are known in advance. We study the offline stage-scheduling problem in exactly this form.

We profile serving windows with the affine model $T(N) = A + B \cdot N$, where A is the fixed window-launch time and B is the marginal cost of adding a request. Across five models, homogeneous stage batches reduce B by 32%–54% relative to heterogeneous batches, while A remains nearly unchanged. The results in vLLM runs also show higher prefix-cache hit rates under stage grouping, indicating that improved prefix locality is an important contributor to the lower marginal cost.

Building on this observation, we formulate stage scheduling for offline typed workflows as a mixed-integer program that assigns ready stages to serving windows and workers subject to workflow precedence, memory capacity, and stage compatibility constraints. Its objective uses an affine serving-time model calibrated from real GPU executions to guide schedule construction. All reported latency results are mea-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

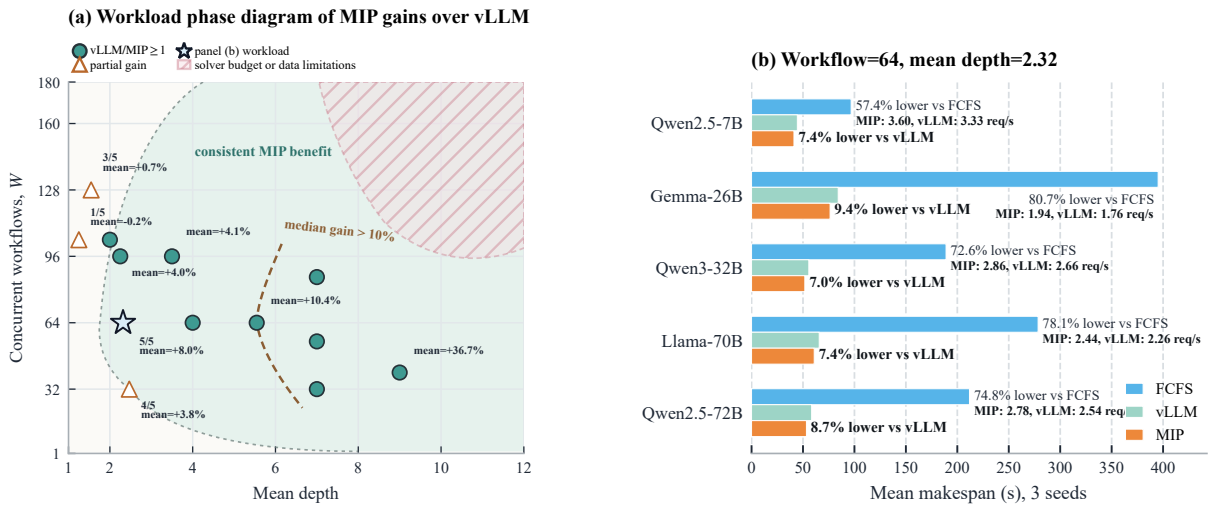


Figure 1: Workload regimes and a representative moderate workload. (a) Measured operating points across mean workflow depth and workflow concurrency W. Circles denote MIP gains over vLLM CB across all five models, whereas triangles indicate gains for a subset. Only markers correspond to measured data. The green envelope and brown contour are schematic, and the hatched region is uncharacterized. All points are from WorkBench except the DeepDAG point at $W = 40$ with mean depth 9, which is included to cover high-depth regimes absent in WorkBench. The star marks the operating point with $W = 64$ and mean depth 2.32. (b) Live wall-clock measurements on the same vLLM engine and matched workflow seeds. At this operating point, MIP reduces mean makespan by 7.0%–9.4% and improves throughput by 7.5%–10.2% relative to vLLM CB across all five models.

sured by executing the resulting schedules on a live vLLM backend.

Across the measured workload regimes, MIP stage scheduling reduces mean makespan by up to 36.7% relative to vLLM continuous batching with automatic prefix caching under synchronous setting. The largest gain occurs on a DeepDAG workload with $W = 40$ and mean depth 9. At a representative moderate workload with W=64 and mean depth 2.32, the reduction is 7.0%–9.4% across all five models, with corresponding throughput gains of 7.5% to 10.2% as shown in Figure 1. The benefit grows substantially on deeper DAG topologies, reaching over 20.9% under synchronous arrivals and up to 47.0% under bursty arrivals. Stage-level grouping consistently outperforms workflow-domain grouping. Real vLLM executions with the WorkBench sandbox evaluator show no systematic degradation in task outcomes.

This paper makes the following contributions.

- We define admission scheduling for offline typed workflows, jointly considering cross-workflow stage compatibility and precedence without using future LLM outputs.
- We formulate a GPU-calibrated MIP that constructs serving windows under precedence and memory constraints, using the calibrated model only for planning and live vLLM execution for evaluation.
- Across five models, we identify a 32%–54% reduction in the marginal cost of homogeneous stage batches and measure 7.0%–9.4% lower makespan than default vLLM at the representative WorkBench point, with gains up to 47.0% in DeepDAG stress tests.

Related Work

Our work lies at the intersection of LLM serving, prefix-aware request scheduling, agent workflow execution, and precedence-constrained scheduling. The closest systems improve prefix locality or exploit workflow information, but generally do not jointly optimize typed-stage admission and batching under workflow precedence constraints.

LLM Serving Systems. Modern LLM serving systems optimize inference at the token, iteration, or request level. ORCA introduces iteration-level scheduling to reduce batch fragmentation (Yu et al. 2022), while vLLM uses PagedAttention and continuous batching for efficient KV-cache management (Kwon et al. 2023). Sarathi-Serve uses chunked prefills to mitigate prefill–decode interference (Agrawal et al. 2024), and DistServe disaggregates the two phases across dedicated resources (Zhong et al. 2024). S-LoRA extends paged serving to large collections of LoRA adapters (Sheng et al. 2023). These systems primarily optimize the execution of requests after admission to the serving engine. Our method instead controls which workflow stages are admitted together, while retaining the underlying engine's token-level execution mechanisms.

Prefix-Aware Scheduling and KV-Cache Reuse. Several systems explicitly exploit shared prompt prefixes. SGLang introduces RadixAttention, which stores reusable KV states in a radix tree and automatically matches incoming requests to cached prefixes (Zheng et al. 2024). Preble extends prefix-aware scheduling to distributed serving and jointly balances computation reuse against load distribution (Srivatsa et al.

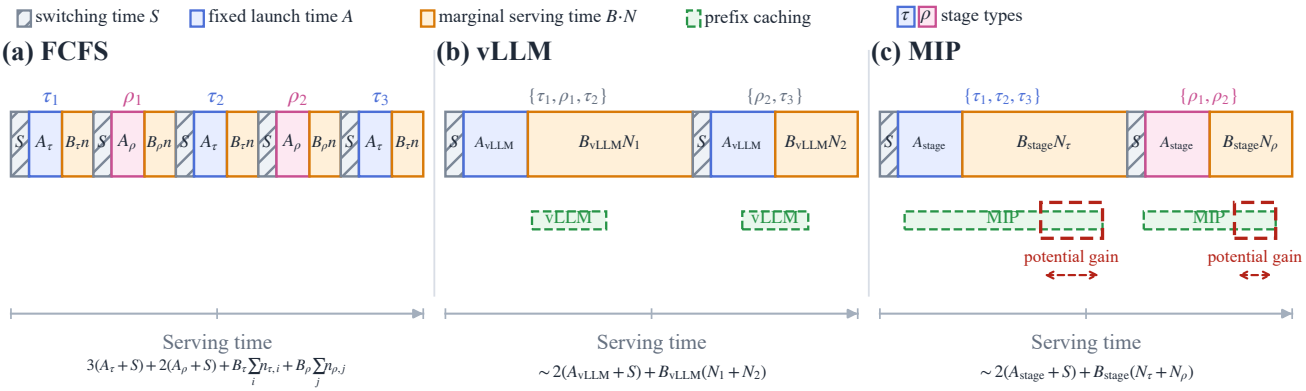


Figure 2: Toy mechanism for stage-aware batching. FCFS places ready stages in separate serving windows, so it pays the fixed launch time A_{stage} for each stage. vLLM continuous batching can mix stage types in one window, which uses the higher marginal cost B_{hetero}. MIP groups ready stages of the same type, pays A_{stage} fewer times, and uses the homogeneous marginal cost B_{stage} due to structurally enforced prefix reuse. The MIP formulation also accounts for switching overhead S, but this term is not the primary source of the observed gains.

2025). BatchLLM targets large offline batches, globally identifies shared prefixes, reorders requests to improve KV reuse, and coordinates prefix sharing with throughput-oriented token batching (Zheng et al. 2026). Recent query-scheduling work formalizes the tradeoff between prefix reuse and latency constraints and proposes k-LPM to balance cache locality with request fairness (Dexter et al. 2025).

These methods are closely related to our use of prefix-compatible grouping. Their scheduling decisions are primarily based on request-level prefix relations, traffic state, or distributed load. Our formulation adds application-level structure that a request belongs to a statically typed workflow stage, becomes available only after its predecessors complete, and must be grouped without violating DAG readiness. We therefore optimize workflow precedence and stage-compatible admission jointly, rather than treating prefix order as an unconstrained permutation of an offline request set.

Agent Orchestration and Workflow-Aware Serving. Agent frameworks coordinate model calls, tools, and dependencies within an application. AutoGen organizes multi-agent interactions through configurable roles and conversation patterns (Wu et al. 2024), while LLMCompiler constructs task DAGs and executes independent tool calls in parallel (Kim et al. 2023). Model-routing methods such as RouteLLM and FrugalGPT select among models to balance quality and cost (Ong et al. 2024; Chen, Zaharia, and Zou 2023). These approaches determine what work should be executed, but do not primarily optimize shared serving windows across multiple workflow instances.

KVFlow represents agent execution using an Agent Step Graph and uses workflow-derived future-use information to guide fine-grained KV-cache eviction and prefetching (Pan et al. 2025). KVFlow and our method exploit complementary forms of workflow information. KVFlow assumes an execution schedule and improves the retention and movement of cached KV states, whereas our method constructs the stage-admission schedule itself by deciding which ready stages should share a serving window. The two mechanisms could be combined, with stage scheduling determining execution order and workflow-aware cache management preserving the KV states likely to be reused by that order.

Scheduling with Precedence Constraints. Precedence constrained scheduling has a long history in operations research and heterogeneous computing. HEFT applies priority-based list scheduling to DAG tasks on heterogeneous processors (Topcuoglu, Hariri, and Wu 2002), and MIP formulations are widely used for NP-hard machine- and job-scheduling problems with precedence constraints (Lenstra, Kan, and Brucker 1977). Classical formulations typically assume that a task's processing time is fixed once its machine is chosen. In our setting, the duration of a serving window depends on the number and compatibility of the stages grouped into it, together with launch and transition overheads. The scheduler must therefore decide both the feasible precedence order and the composition of each batch. This batch dependent duration distinguishes our formulation from standard DAG list scheduling and job-shop models.

Problem Formulation

We propose a mixed integer programming formulation for agent serving workflows. The model schedules workflow stages into discrete serving windows and groups compatible stages to amortize batch serving time under the workflow and per-worker resource constraints stated below.

Sets and Parameters. Table 1 summarizes the notation used in the formulation.

Objective. The main objective minimizes the makespan:

$$\min \; C_{\max}. \tag{1}$$

The same formulation also supports weighted completion-time variants for priority-aware serving (See supplementary Section 4).

Table 1: Notation for the stage-aware scheduling formulation.

Sets	
$\mathcal{R}$	Set of unfinished stages in current planning horizon.
$\mathcal{K}$	Set of stage types.
$\mathcal{M}$	Set of GPU workers.
$\mathcal{H} = \{1, \dots, H\}$	Serving windows on each worker.
$\mathcal{E}$	Workflow precedence edges. An edge (r_1, r_2) means that stage r_1 must finish before stage r_2 starts.
Parameters	
R_r	Release time of stage r.
$K(r)$	Computational type of stage r.
F_{km}	fixed window-launch time for serving type k on worker m.
P_{rm}	Profiled marginal runtime coefficient of stage r on worker m.
S_{km}	Nonnegative setup/switching time when worker m changes to stage type k.
V_{rm}	Memory footprint of stage r on worker m.
$\mathcal{A}_{km}$	Adapter or type-specific memory cost of type k on worker m.
B_m	Base model memory on worker m.
$\bar{M}_m$	Memory capacity of worker m.
L_{km}	Type weight used by the grouping constraint.
Γ_m	Grouping budget for worker m. Setting $\Gamma_m = 1$ enforces one active type per window.
G^0_{km}	Indicator of whether type k is already loaded on worker m at the start of the planning horizon.
U	A sufficiently large constant used in big-M constraints.
Decision Variables	
$x_{rhm} \in \{0, 1\}$	Equals one if stage r is assigned to window h on worker m.
$g_{khm} \in \{0, 1\}$	Equals one if stage type k is active in window h on worker m.
$u_{khm} \in \{0, 1\}$	Equals one if stage type k is newly loaded in window h on worker m.
$t^s_{hm} \in \mathbb{R}_+$	Start time of window h on worker m.
$t^e_{hm} \in \mathbb{R}_+$	End time of window h on worker m.
$C_{\max} \in \mathbb{R}_+$	Makespan of the planning horizon.

Constraints. The formulation is subject to the following constraints. Unless otherwise specified, indices range over their corresponding sets.

$$\sum_{m \in \mathcal{M}} \sum_{h \in \mathcal{H}} x_{rhm} = 1, \quad \forall r \in \mathcal{R}. \tag{2}$$

$$t^s_{hm} \geq R_r - U(1 - x_{rhm}), \quad \forall r, h, m. \tag{3}$$

$$x_{rhm} \leq g_{K(r),h,m}, \quad \forall r, h, m. \tag{4}$$

$$\sum_{k \in \mathcal{K}} L_{km} g_{khm} \leq \Gamma_m, \quad \forall h, m. \tag{5}$$

$$u_{k,1,m} \geq g_{k,1,m} - G^0_{km}, \quad \forall k, m. \tag{6}$$

$$u_{k,h,m} \geq g_{k,h,m} - g_{k,h-1,m}, \quad \forall k, m,\ h = 2, \dots, H. \tag{7}$$

$$B_m + \sum_k \mathcal{A}_{km} g_{khm} + \sum_r V_{rm} x_{rhm} \leq \bar{M}_m, \quad \forall h, m. \tag{8}$$

$$t^e_{hm} - t^s_{hm} \geq \sum_{k \in \mathcal{K}} F_{km} g_{khm} + \sum_{r \in \mathcal{R}} P_{rm} x_{rhm} + \sum_{k \in \mathcal{K}} S_{km} u_{khm}, \quad \forall h, m. \tag{9}$$

$$t^s_{h+1,m} \geq t^e_{hm}, \quad \forall m,\ h = 1, \dots, H - 1. \tag{10}$$

$$t^e_{h_1 m_1} \leq t^s_{h_2 m_2} + U(2 - x_{r_1 h_1 m_1} - x_{r_2 h_2 m_2}), \quad \forall (r_1, r_2) \in \mathcal{E},\ h_1, h_2 \in \mathcal{H},\ m_1, m_2 \in \mathcal{M}. \tag{11}$$

$$C_{\max} \geq t^e_{hm}, \quad \forall h, m. \tag{12}$$

Constraints (2)–(5) assign each stage to one worker and serving window, respect release times, and enforce the grouping budget. In the homogeneous setting used in the main experiments, $\Gamma_m = 1$, so each window contains at most one stage type. Constraints (6) and (7) identify a newly activated type, while Constraint (8) enforces the worker memory limit. Constraint (9) combines the fixed window-launch time, marginal serving time, and switching term S_{km}. Constraints (10)–(12) order serving windows, preserve workflow precedence, and define the planning-horizon makespan. Supplementary Section 1 provides additional notation and details on readiness and precedence constraints, memory accounting, and auxiliary setup terms. The pseudocode of the runtime scheduling protocol is presented in Algorithm 1.

Batching Mechanism. With $\Gamma_m = 1$, a window containing N stages of one type has profiled duration

$$A_{\text{stage}} + B_{\text{stage}} N + S,$$

where S applies only when the type changes. FCFS repeatedly incurs the fixed term, whereas heterogeneous batching

Algorithm 1: Stage-Aware Scheduling Protocol

Input: Workflow set $\mathcal{W}$, arrival pattern, horizon H
Initialize: $G^0_{km} \leftarrow 0$ for all k, m; inject arrivals into system
while unfinished stages exist **do**
  $\mathcal{R} \leftarrow$ unfinished stages in current planning horizon
  if $\mathcal{R} \neq \emptyset$ **then**
    Solve MIP over $\mathcal{R}$ for H windows per worker
    Execute all planned windows
    Mark completed stages; update release times of successors
    Update $G^0_{km} \leftarrow g_{kHm}$ (persist loaded types)
  end if
  Process any new arrivals (Poisson/bursty) into workflow pool
end while
Return: Measured makespan from first admission to last completion

uses the larger profiled coefficient B_{hetero}. Stage grouping amortizes the fixed term and uses B_{stage} for compatible requests. Completed stages release their successors, after which the model is re-solved over the updated unfinished set.

Serving abstraction. The MIP models an external admission layer above vLLM rather than the engine's internal token-level continuous batching process. A serving window specifies which ready requests are jointly admitted, while vLLM retains control of prefill, decode, and iteration-level execution within that window. The affine duration model is therefore a planning-time approximation of the observed wall-clock cost of an admitted request group, not an exact model of vLLM's internal scheduler.

Experimental Setup

Our evaluation separates schedule construction from schedule measurement. We first profile real vLLM executions to calibrate the affine serving-time parameters used by the MIP. These calibrated parameters are used only to construct a schedule. For each policy, we then execute the resulting schedule on the same real vLLM engine using matched workflow instances and random seeds. We use matched workflow instances, prompts, model checkpoints, decoding configuration, and arrival patterns, changing only the admission order and grouping of ready stages. Reported makespan is measured wall-clock time from first request to final completion. For task behavior, we separately run matched WorkBench workflows through the live agent executor and sandbox evaluator, holding all parameters fixed except serving order. In our offline execution pipeline, the initial schedule is constructed before serving begins, while subsequent CPU-side MIP solves are overlapped with GPU execution of the current serving windows.Each MIP solve completes within a few seconds for the reported workloads, and we observe no additional critical-path delay from replanning. We report the raw solve-time distribution and planning-round statistics separately in the supplementary material.

Model Calibration. We profile heterogeneous and homogeneous batches on NVIDIA H100 80GB GPUs using vLLM. Heterogeneous batches mix stage types, whereas homogeneous batches contain a single stage type. For each regime, we measure end-to-end serving time at 27 nonuniform batch sizes up to $N = 256$, with three repetitions per size. The grid is denser for $N \leq 96$, where the scheduler operates most often. Here, N denotes the number of stage requests in one serving window. We fit $T(N) = A + B \cdot N$ by ordinary least squares, where A is the fixed window-launch time and B is the marginal serving time. The MIP uses only the coefficients calibrated from homogeneous stage batches to construct its serving windows. The heterogeneous fit is reported solely as a profiling reference for mixed-stage batching and does not enter the MIP objective, constraints, or schedule construction. All headline latency results are measured by executing the resulting schedules on the real vLLM engine. The fitted value $T(N)$ is not used as the reported makespan.

Table 2 reports the fitted parameters for five models from 7 B to 72 B. Gemma-26B refers to the Gemma-4-26B-A4B-it MoE checkpoint (Gemma Team 2026). The table reports the nominal total parameter scale, and all comparisons use the measured serving profile for that checkpoint. The two profiling regimes use the same request multiset and matched prompt and output token distributions. Across a complete profiling run, the request multiset is identical. Only the order and stage composition of each submitted batch differ. Across all five models, $A_{\text{hetero}} \approx A_{\text{stage}}$ within ± 0.12 s, indicating that the fixed window-launch cost is insensitive to batch composition. The marginal cost B, however, is $1.5\times$ to $2.2\times$ higher in the heterogeneous regime which is consistent with improved prefix-cache reuse. The supplementary material reports profiling details, fit diagnostics and transition-term sensitivity checks.(See Supplementary Section 2 & Section 3)

Model	Hetero $A(s)$	Hetero $B(ms/it)$	Stage $A(s)$	Stage $B(ms/it)$	R^2_H	R^2_S
Qwen2.5-7B	0.592	6.094	0.603	3.196	0.836	0.950
Gemma-26B	2.580	17.129	2.632	11.748	0.829	0.944
Qwen3-32B	1.204	10.345	1.238	6.517	0.846	0.951
Llama-70B	1.951	19.692	1.835	9.106	0.833	0.931
Qwen2.5-72B	1.371	11.042	1.391	7.084	0.841	0.926

Table 2: Affine serving-time parameters $T(N) = A + B \cdot N$ fitted from vLLM profiling under heterogeneous and homogeneous batching regimes, using 27 nonuniform batch sizes from $N = 1$ to 256. Heterogeneous batches mix stage types, whereas homogeneous batches contain a single stage type. Across five models, the fixed cost A remains nearly unchanged, while the marginal cost B drops by 32%–54% under homogeneous grouping, consistent with improved prefix-cache reuse.

Workloads. WorkBench (Styles et al. 2024) provides workplace tasks with sandbox-evaluated outcomes. Before scheduled execution, each public action trace is compiled into a static typed workflow whose stage types, prompt templates, and predecessor relations are fixed and shared by all policies. At runtime, the scheduler observes only the compiled graph and currently ready nodes. It does not use future model outputs or recover stage labels online. Because the

traces do not expose reliable parallel dependencies, we conservatively materialize them as sequential chains.

DeepDAG is a controlled synthetic workload with predefined branching and joining templates that are difficult to recover from public action logs (Styles et al. 2024; Li et al. 2025; Luo et al. 2025; Li et al. 2026). Its nodes represent planning, retrieval, reasoning, acting, verification, and summarization stages.

Scheduling Policies. We compare three scheduling policies on identical workloads using the same vLLM serving engine, model weights, prompts, and tools. A widely used serving baseline vLLM Continuous Batching forms batches internally with automatic prefix caching without any explicit grouping by stage type. FCFS is a naive serial reference that disables cross-request batching and isolates the aggregate benefit of batching and prefix-aware admission. MIP Stage scheduling assigns ready stages to serving windows and workers under precedence, memory, and switching constraints. It groups compatible stages within each window and submits them to vLLM in one dispatch, allowing the prefix cache to exploit their shared structure.

We also include a grouping ablation motivated by the structure of WorkBench, where tasks naturally belong to workplace domains labels provided by WorkBench. Stage-level grouping uses the typed computational stage of each ready request. Mode-level grouping instead uses the workflow-level domain label in WorkBench and the workflow family label in DeepDAG.

Metrics. The primary latency metric is makespan. For task behavior, we report evaluator outcomes from real vLLM runs with the WorkBench sandbox evaluator. Since the scheduler preserves all workflow parameters except serving order and grouping, this evaluation isolates the scheduling effect.

Experimental Results

Fixed-Model Gains

We first examine the within-model gains of stage scheduling. Figure 1(a) summarizes the measured operating points across workflow concurrency and mean depth. MIP Stage Scheduling achieves up to 36.7% lower makespan than vLLM CB across these points. To assess whether the benefit holds under a moderate workload, we focus on the representative operating point marked by the star in Figure 1(b), where $W{=}64$ and the mean depth is 2.32. In this controlled setup, policies differ solely in how they order and group requests. At this operating point, stage scheduling reduces mean makespan by 57.4% to 80.7% relative to FCFS and by 7.0% to 9.4% relative to vLLM CB with automatic prefix caching. The improvement is consistent across Qwen, Gemma, and Llama models ranging from 7 B to 72 B, indicating that the benefit is not tied to a particular model family or scale. These results motivate a closer look at prefix locality under stage grouping.

Prefix Locality Under Stage Grouping

To verify whether stage grouping improves prefix locality in vLLM, we run controlled telemetry experiments with automatic prefix caching enabled. For each model and request-set size $N \in [2, 1024]$, we execute the same request set under default vLLM dispatch and stage-aware strategy, measuring prefix-cache hit rate from vLLM internal counters. We report the mean and standard deviation across three random seeds.

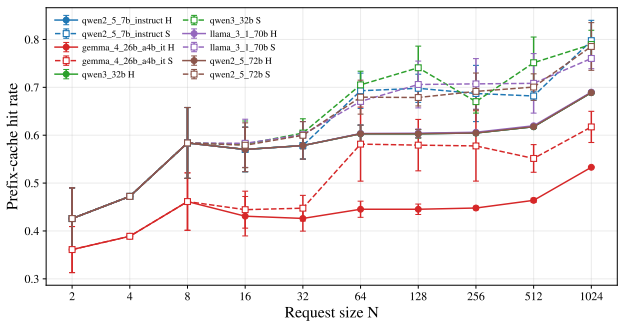


Figure 3: Prefix-cache hit rate versus request-set size N across five models. H denotes default vLLM dispatch, and S denotes the stage-aware scheduling strategy. Hit rate is computed from vLLM counter deltas as $\#\text{hits}/\#\text{queries}$. Markers and error bars report the mean and one standard deviation over three seeds. For $N \geq 64$, stage grouping improves hit rate by 6.42–13.79 percentage points.

Figure 3 shows that stage grouping requires a sufficient number of ready stages to outperform default vLLM. The prefix cache benefit emerges at a threshold of $N = 16$ for larger models (26B–72B) and $N = 64$ for Qwen-7B. For $N \geq 64$, stage grouping increases the prefix-cache hit rate by 6.4%–13.8%, whereas at $N \leq 8$, the hit rates are nearly identical and grouping overhead dominates.

These trends support prefix locality as one contributor to the lower marginal serving time. We next examine which workload properties provide such grouping opportunities.

When Stage Scheduling Helps Most

The prefix-locality experiments suggest that stage grouping becomes useful when a sufficiently dense pool of compatible ready requests is available. We therefore examine how workflow topology and arrival pattern affect these opportunities. Figure 4 compares MIP-guided scheduling with vLLM continuous batching under synchronous, Poisson, and bursty arrivals. On the shallow WorkBench chains, MIP reduces makespan by 10.9%–15.5%. On the deeper branch-and-join DeepDAG workload, the reduction is 20.9% under synchronous arrivals and 47.0% under both the tested Poisson and bursty arrivals. These results indicate that the benefit increases when branching structure and arrival dynamics create denser ready-stage pools. Because WorkBench and DeepDAG differ in both topology and request composition, this comparison is interpreted as a workload-level stress test rather than a topology-only causal ablation.

Stage Abstraction Matters for Scheduling

The previous sections used stage-level grouping. A natural alternative is to group by the user-facing workflow mode, such as email, calendar, CRM, analytics, project management, or

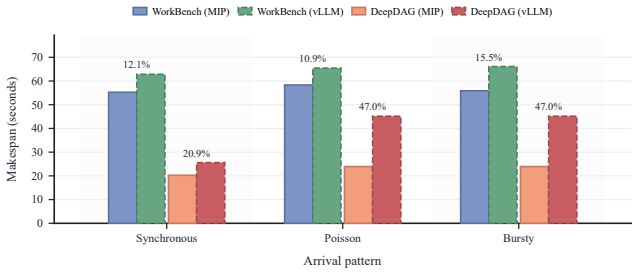


Figure 4: Robustness across topologies and arrival patterns. WorkBench (96 workflows, 408 requests, sequential chains) versus DeepDAG (15 workflows, 405 requests, branching/joining DAGs). MIP vs. vLLM-CB, 3 seeds.

the workflow family in DeepDAG. Figure 5 compares the two under synchronous arrivals with the same workloads, models, and scheduler. Stage-level grouping reduces makespan by 7.6%–28.9% relative to mode grouping across all five models and both workflow counts. This shows that the serving layer benefits from a finer computational abstraction than the user-facing task category.

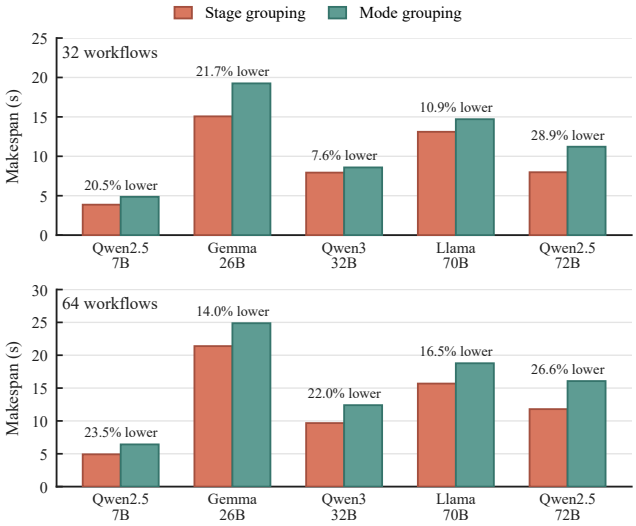


Figure 5: Stage-level versus mode-level grouping under synchronous WorkBench arrivals. Percentage labels report the relative makespan reduction of stage grouping over mode grouping, $(T_{\text{mode}} - T_{\text{stage}})/T_{\text{mode}}$.

The reason is that mode and stage capture different signals. A mode label describes the user-facing task category, but often mixes planning, tool use, verification, and synthesis stages. A stage is closer to the computation being served, so it exposes more homogeneous batching opportunities and better matches the affine serving-window model. This finer abstraction can therefore better exploit low marginal serving time within a window. The tradeoff is that stage-level grouping expands the solver search space, giving the scheduler more assignment variables and type-activation constraints to optimize, while mode grouping keeps a smaller search space. Figure 5 shows that this coarser abstraction leaves serving efficiency on the table in the tested WorkBench settings.

Task Behavior Under Reordered Ready Stages

Model	Policy	Comp.	Corr./comp.	Err.
Qwen2.5-7B	FCFS	43.7±3.2	27.8%	0.0%
Qwen2.5-7B	Stage	42.3±3.8	27.9%	0.0%
Qwen2.5-72B	FCFS	41.0±3.6	30.9%	0.0%
Qwen2.5-72B	Stage	39.7±1.2	31.9%	0.0%
Qwen3-32B NT	FCFS	54.7±3.5	46.2%	0.0%
Qwen3-32B NT	Stage	56.0±2.6	47.0%	0.0%
Qwen3-32B T	FCFS	19.0±4.0	78.1%	0.0%
Qwen3-32B T	Stage	18.0±3.5	84.5%	0.0%

Table 3: Real-execution WorkBench task-behavior check over three matched seeds. Stage denotes MIP Stage Scheduling. NT and T denote no-thinking and thinking variants of Qwen3-32B. Each seed contains 60 workflows. Comp. reports the mean number of completed tasks. Corr./comp. reports the mean evaluator accuracy among completed tasks, Err. reports the mean sandbox error rate. Supplementary Section 5 reports full statistics.

Table 3 reports matched WorkBench sandbox executions. Across the four model conditions, stage scheduling shows no consistent directional change in completion or conditional evaluator accuracy, and all reported sandbox error rates are zero. The lower completion of Qwen3-32B-T reflects its greater token and iteration use under the fixed sandbox budget. Detailed outcome statistics are provided in Supplementary Section 5.

Conclusion

This paper identifies a missing layer in agent serving between agent orchestration and token execution. To address this gap, we introduce an admission scheduler for offline typed agent workflows that groups compatible ready stages across workflows while preserving precedence constraints. Across five models, MIP-guided scheduling reduces makespan in live serving experiments by 7.0%–9.4% relative to vLLM continuous batching at a representative WorkBench operating point and by up to 47.0% in DeepDAG stress tests. Homogeneous stage batches exhibit a 32%–54% lower marginal serving-time coefficient, while cache logs indicate that improved prefix locality is one contributing mechanism. Importantly, we observe no statistically significant degradation in task outcomes under the tested WorkBench settings. Future work will extend the approach to online arrivals, dynamically generated workflows, and policies that respond to deadlines and system state. Another important extension is to couple stage-level scheduling with token-level execution, where prefill and decode phases exhibit distinct serving-time structures not captured by a single affine model. Unifying these perspectives would connect workflow-level orchestration, stage-level scheduling, and token-level batching into a more complete model of end-to-end agent serving.

References

Agrawal, A.; Kedia, N.; Panwar, A.; Mohan, J.; Kwatra, N.; Gulavani, B. S.; Tumanov, A.; and Ramjee, R. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation, 2024, Santa Clara*.

Chen, L.; Zaharia, M.; and Zou, J. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.

Dexter, G.; Tang, S.; Fatahi, A.; Song, Q.; Dharamsi, T.; and Gupta, A. 2025. LLM Query Scheduling with Prefix Reuse and Latency Constraints. In *Advances in Neural Information Processing Systems*.

Gemma Team, G. D. 2026. Gemma 4. https://ai.google.dev/gemma/docs/core/model_card_4. Instruction-tuned multimodal models.

Kim, S.; Moon, S.; Tabrizi, R.; Lee, N.; Mahoney, M. W.; Keutzer, K.; and Gholami, A. 2023. An llm compiler for parallel function calling. *arXiv preprint arXiv:2312.04511*.

Kwon, W.; Yin, S.; Han, J.; Chen, J.; Lo, B.; et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626.

Lenstra, J. K.; Kan, A. R.; and Brucker, P. 1977. Complexity of machine scheduling problems. In *Annals of discrete mathematics*, volume 1, 343–362. Elsevier.

Li, J.; Zhao, W.; Zhao, J.; Zeng, W.; Wu, H.; Wang, X.; Ge, R.; Cao, Y.; Huang, Y.; Liu, W.; Liu, J.; Su, Z.; Guo, Y.; Zhou, F.; Zhang, L.; Michelini, J.; Wang, X.; Yue, X.; Zhou, S.; Neubig, G.; and He, J. 2025. The Tool Decathlon: Benchmarking Language Agents for Diverse, Realistic, and Long-Horizon Task Execution. *arXiv preprint arXiv:2510.25726*.

Li, K.; Shi, J.; Xiao, Y.; Jiang, M.; Sun, J.; Wu, Y.; Xia, S.; Cai, X.; Xu, T.; Si, W.; et al. 2026. AgencyBench: Benchmarking the Frontiers of Autonomous Agents in 1M-Token Real-World Contexts. *arXiv preprint arXiv:2601.11044*.

Luo, Z.; Yuan, C.; Tian, X.; Wang, Y.; Shi, J.; et al. 2025. MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers. arXiv:2508.14704.

Ong, I.; Almahairi, A.; Wu, V.; Chiang, W.-L.; Wu, T.; Gonzalez, J. E.; Kadous, M. W.; and Stoica, I. 2024. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*.

Pan, Z.; Patel, A.; Shen, Y.; Hu, Z.; Guan, Y.; Li, W.-L.; Qin, L.; Wang, Y.; and Ding, Y. 2025. KVFlow: Efficient Prefix Caching for Accelerating LLM-Based Multi-Agent Workflows. In *Advances in Neural Information Processing Systems*.

Sheng, Y.; Zheng, L.; Zhu, B.; Li, Z.; Zhang, D.; et al. 2023. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. arXiv:2311.03285.

Srivatsa, V.; He, Z.; Abhyankar, R.; Li, D.; and Zhang, Y. 2025. Preble: Efficient Distributed Prompt Scheduling for Large Language Model Serving. In *International Conference on Learning Representations*.

Styles, O.; Miller, S.; Cerda-Mardini, P.; Guha, T.; Sanchez, V.; and Vidgen, B. 2024. Workbench: a benchmark dataset for agents in a realistic workplace setting. *arXiv preprint arXiv:2405.00823*.

Topcuoglu, H.; Hariri, S.; and Wu, M.-Y. 2002. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3): 260–274.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First conference on language modeling*.

Yu, G.-I.; Jeong, J. S.; Kim, G.-W.; Kim, S.; and Chun, B.-G. 2022. ORCA: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 521–538. USENIX Association.

Zheng, L.; Yin, L.; Xie, Z.; Sun, C.; Huang, J.; Yu, C. H.; Cao, S.; Kozyrakis, C.; Stoica, I.; Gonzalez, J. E.; Barrett, C.; and Sheng, Y. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems*, volume 37.

Zheng, Z.; Ji, X.; Fang, T.; Zhou, F.; Liu, C.; and Peng, G. 2026. BatchLLM: Optimizing Large Batched LLM Inference with Global Prefix Sharing and Throughput-oriented Token Batching. In *Ninth Conference on Machine Learning and Systems*.

Zhong, Y.; Liu, S.; Chen, J.; Hu, J.; Zhu, Y.; Liu, X.; Jin, X.; and Zhang, H. 2024. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 193–210.